

LAPORAN BAB 5
Praktikum Devops
Database Service di Docker: PostgreSQL



Disusun Oleh:
ARSYITA DEVANAYA ARIANTO
NRP. 3126640046

PROGRAM REKOGNISI PEMBELAJARAN LAMPAU (RPL)
JURUSAN TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
SURABAYA
2026

1. Tujuan dan Ruang Lingkup

Praktikum Bab 5 bertujuan untuk:

1. Menjalankan PostgreSQL dan pgAdmin sebagai container dengan konfigurasi berbasis environment variable dan volume persisten.
2. Menginisialisasi database menggunakan file SQL pada `docker-entrypoint-initdb.d`.
3. Mengakses database melalui `psql` dan pgAdmin4.
4. Melakukan logical backup menggunakan `pg_dump` beserta checksum, lalu menguji restore menggunakan `pg_restore` pada database terpisah.
5. Menerapkan healthcheck pada service PostgreSQL dan memverifikasi persistensi data volume.

Ruang lingkup bab ini mencakup pembuatan struktur project `~/docker-lab/bab-5` berisi Compose stack PostgreSQL–pgAdmin, init script, konfigurasi server pgAdmin (`servers.json`), script backup dan restore test, pengujian persistensi volume, verifikasi checksum dump, serta cleanup yang aman tanpa menghapus data.

2. Landasan Teori

2.1 Manajemen Database Container dan Persistensi Volume

PostgreSQL di dalam container tetap merupakan sistem database relasional penuh; container hanya mengubah cara proses dan dependensinya dikemas. Data tidak boleh dianggap melekat pada umur container. Writable layer sesuai untuk perubahan sementara, sedangkan data database memerlukan **volume persisten** yang dapat dipasang kembali ketika container diganti.

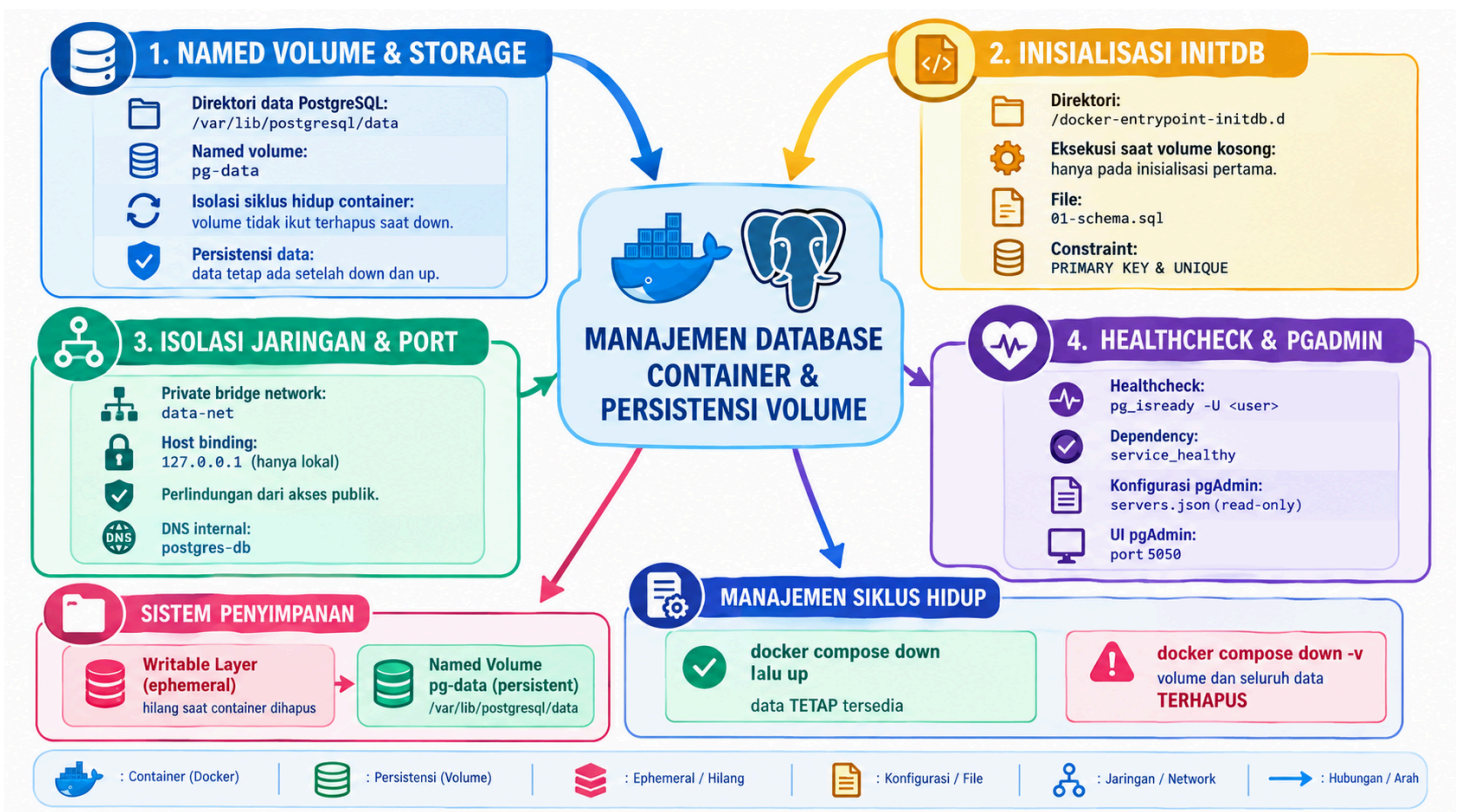
Image PostgreSQL resmi mengeksekusi file `.sql` dan `.sh` di `/docker-entrypoint-initdb.d` **hanya saat volume data masih kosong**. Mengubah init script tidak akan berpengaruh bila volume lama masih dipakai. Init script bukan pengganti migration untuk perubahan skema berikutnya.

Variabel	Fungsi
POSTGRES_DB	Nama database awal yang dibuat ketika inisialisasi
POSTGRES_USER	User awal/superuser container
POSTGRES_PASSWORD	Password awal untuk user tersebut
PGADMIN_DEFAULT_EMAIL	Email login administratif pgAdmin

Variabel	Fungsi
PGADMIN_DEFAULT_PASSWORD	Password login administratif pgAdmin

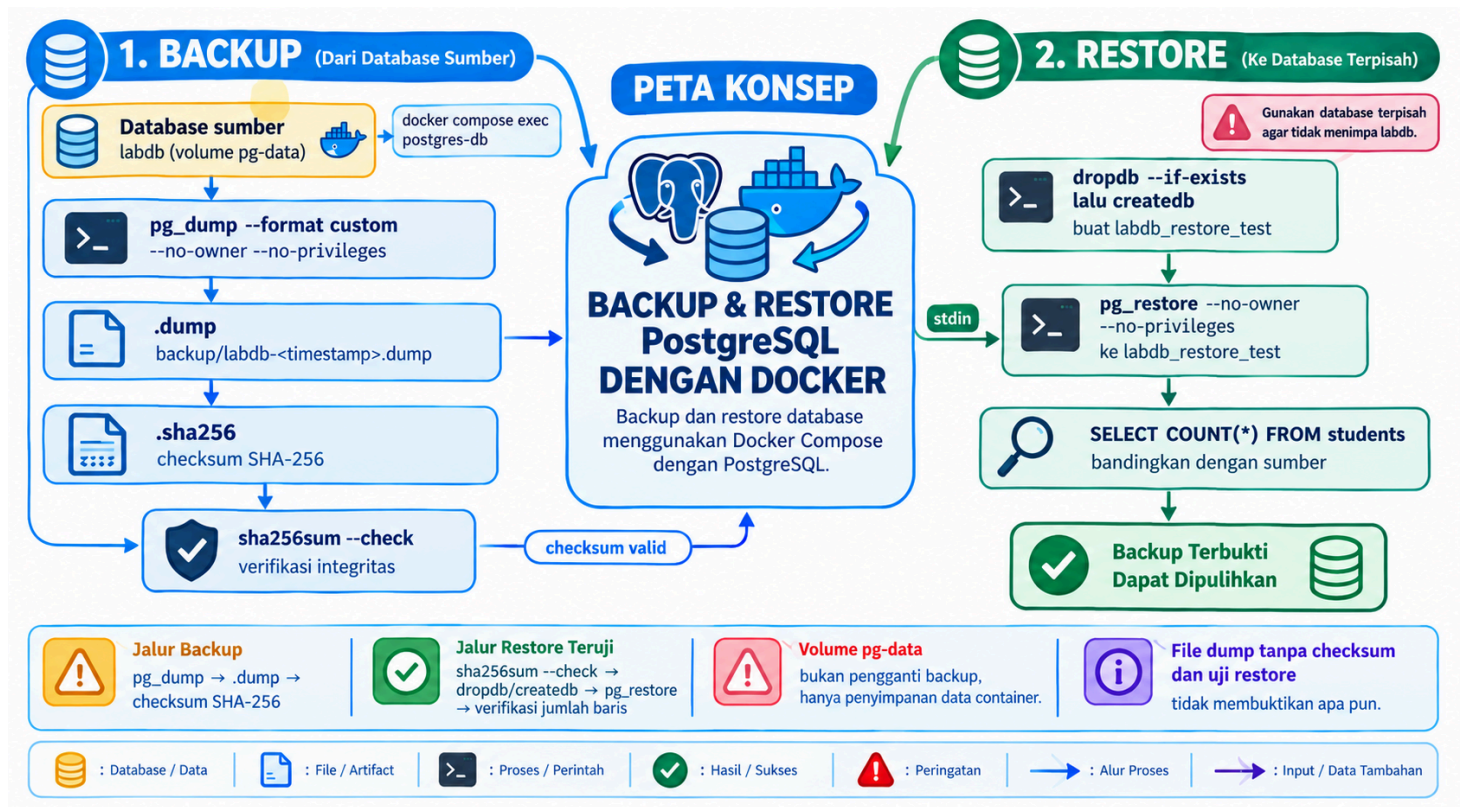
Persistensi data memiliki empat dimensi: durability transaksi, keberlangsungan media, kemampuan backup, dan kemampuan **restore**. Credential tidak boleh ditanam pada image atau repository; dalam laboratorium `.env` dapat dipakai untuk demonstrasi, tetapi file tersebut harus dikecualikan dari version control dan tidak diperlakukan sebagai secret manager produksi. Port PostgreSQL dan pgAdmin pada praktikum ini dibatasi ke loopback `127.0.0.1`. Healthcheck `pg_isready` hanya membuktikan server menerima koneksi, bukan bahwa migrasi atau query bisnis benar.

2.2 Peta Konsep: Manajemen Database Container dan Persistensi Volume



Peta konsep di atas memetakan hubungan antara container, writable layer yang bersifat sementara, dan named volume `pg-data` yang menyimpan data secara persisten. Init script pada `/docker-entrypoint-initdb.d` hanya dibaca sekali, yaitu ketika volume masih kosong, sehingga perubahan skema berikutnya harus dikelola lewat migration. Karena data hidup pada volume — bukan pada container — perintah `docker compose down` yang diikuti `up` tidak menghilangkan data, sedangkan `docker compose down -v` bersifat destruktif dan menghapus volume beserta seluruh isinya.

2.3 Peta Konsep: Backup dan Restore PostgreSQL dengan Docker



Peta konsep di atas memetakan dua jalur utama, yaitu backup (`pg_dump` → file `.dump` → checksum SHA-256) dan restore (`sha256sum --check` → `dropdb/createdb` → `pg_restore` → verifikasi jumlah baris). Restore sengaja diarahkan ke database terpisah `labdb_restore_test` agar tidak menimpa `labdb`. Volume `pg-data` bukan pengganti backup, dan file dump tanpa verifikasi checksum serta uji restore tidak membuktikan apa pun.

3. Metodologi

Laboratorium menggunakan satu host yang menjalankan Docker Engine dan Docker Compose v2 dengan direktori kerja `~/docker-lab/bab-5` yang berisi `init`, `pgadmin`, `scripts`, dan `backup`. Konfigurasi ditulis sebagai file di host: `.env.example` (template) disalin menjadi `.env` dengan permission 600, `compose.yaml` mendefinisikan `postgres-db` dan `pgadmin` pada network `data-net` dengan dua named volume, dan `init/01-schema.sql` menyiapkan tabel `students` beserta data awal.

Verifikasi dilakukan berurutan: validasi model Compose (docker compose config), pemeriksaan status dan health, pengujian schema/data awal lewat `psql`, pengujian persistensi volume dengan `down-up`, pembuatan backup dan verifikasi checksum, uji restore ke `labdb_restore_test`, monitoring dasar, lalu cleanup yang mempertahankan data. Setiap sub-langkah diuji sebelum melanjutkan ke langkah berikutnya.

4. Pelaksanaan Praktikum

4.1 Direktori Kerja

Membuat struktur direktori kerja proyek `~/docker-lab/bab-5` beserta subdirektornya (`init` , `backup` , `pgadmin` , `scripts`), lalu pindah ke direktori tersebut.

```
mkdir -p ~/docker-lab/bab-5/{init,backup,pgadmin,scripts}
cd ~/docker-lab/bab-5
```

```
ubuntu123@DESKTOP-7A8ST00:~$ mkdir -p ~/docker-lab/bab-5/{init,backup,pgadmin,scripts}
ubuntu123@DESKTOP-7A8ST00:~$ cd ~/docker-lab/bab-5
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ tree
.
├── backup
├── init
├── pgadmin
└── scripts

5 directories, 0 files
```

Struktur file yang disusun pada bab ini adalah sebagai berikut.

```
bab-5/
├── .env.example
├── .env
├── .gitignore
├── compose.yaml
├── init/
│   └── 01-schema.sql
├── pgadmin/
│   └── servers.json
├── scripts/
│   ├── backup.sh
│   └── restore-test.sh
└── backup/
```

4.2 File `.env.example` dan `.env`

Menulis `.env.example` sebagai template konfigurasi.

```
nano .env.example
```

```
POSTGRES_DB=labdb
POSTGRES_USER=labuser
POSTGRES_PASSWORD=labpass123
PGADMIN_DEFAULT_EMAIL=admin@example.com
PGADMIN_DEFAULT_PASSWORD=admin123
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano .env.example
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat .env.example
POSTGRES_DB=labdb
POSTGRES_USER=labuser
POSTGRES_PASSWORD=labpass123
PGADMIN_DEFAULT_EMAIL=admin@example.com
PGADMIN_DEFAULT_PASSWORD=admin123
```

Menyalin template menjadi konfigurasi aktif, lalu mengunci permission file agar tidak terbaca service lain.

```
cp .env.example .env
chmod 600 .env
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cp .env.example .env
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ chmod 600 .env
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ ls -la
total 32
drwxr-xr-x 6 ubuntu123 ubuntu123 4096 Oct  8 19:47 .
drwxr-xr-x 6 ubuntu123 ubuntu123 4096 Oct  8 19:36 ..
-rw----- 1 ubuntu123 ubuntu123  145 Oct  8 19:47 .env
-rw-r--r-- 1 ubuntu123 ubuntu123  145 Oct  8 19:44 .env.example
drwxr-xr-x 2 ubuntu123 ubuntu123 4096 Oct  8 19:36 backup
drwxr-xr-x 2 ubuntu123 ubuntu123 4096 Oct  8 19:36 init
drwxr-xr-x 2 ubuntu123 ubuntu123 4096 Oct  8 19:36 pgadmin
drwxr-xr-x 2 ubuntu123 ubuntu123 4096 Oct  8 19:36 scripts
```

4.3 File .gitignore

Menulis .gitignore untuk mencegah credential dan hasil dump ikut terkirim ke Git.

```
nano .gitignore
```

```
.env
backup/*.dump
backup/*.sha256
backup/*.log
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano .gitignore
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat .gitignore
.env
backup/*.dump
backup/*.sha256
backup/*.log
```

4.4 File compose.yaml

Menulis `compose.yaml` yang berisi 2 service (`postgres-db` dan `pgadmin`), 2 named volume (`pg-data` , `pgadmin-data`), 1 network bridge `data-net` , dan healthcheck `pg_isready` pada PostgreSQL. Port kedua service dibatasi ke loopback `127.0.0.1` .

```
nano compose.yaml
```

```
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - data-net
    healthcheck:
      test:
        - CMD-SHELL
        - "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
    restart: unless-stopped
```

```
pgadmin:
  image: dpage/pgadmin4:latest
  environment:
    PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
    PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
  ports:
    - "127.0.0.1:5050:80"
  volumes:
    - pgadmin-data:/var/lib/pgadmin
    - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
  networks:
    - data-net
  depends_on:
    postgres-db:
      condition: service_healthy
  restart: unless-stopped
```

```
volumes:
  pg-data:
```

pgadmin-data:

networks:

data-net:

driver: bridge

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano compose.yaml
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat compose.yaml
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - data-net
    healthcheck:
      test:
        - CMD-SHELL
        - "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
    restart: unless-stopped

  pgadmin:
    image: dpage/pgadmin4:latest
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "127.0.0.1:5050:80"
    volumes:
      - pgadmin-data:/var/lib/pgadmin
      - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
    networks:
      - data-net
    depends_on:
      postgres-db:
        condition: service_healthy
    restart: unless-stopped

volumes:
  pg-data:
  pgadmin-data:

networks:
  data-net:
    driver: bridge
```

`./init` dipasang sebagai mount read-only pada `/docker-entrypoint-initdb.d` sehingga init script hanya dibaca container. pgAdmin menunggu PostgreSQL healthy melalui `depends_on` dengan `condition: service_healthy`. Volume `pg-data` memisahkan data dari lifecycle container.

4.5 File `init/01-schema.sql`

Menulis init script yang membuat tabel `students` dengan kolom `id` (`BIGSERIAL`, primary key), `nrp` (unique), `name`, dan `created_at` (timestamp UTC), lalu mengisinya dua baris data awal secara idempotent dengan `ON CONFLICT DO NOTHING`.

```
nano init/01-schema.sql
```

```
CREATE TABLE IF NOT EXISTS students (  
    id BIGSERIAL PRIMARY KEY,  
    nrp VARCHAR(20) UNIQUE NOT NULL,  
    name TEXT NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
INSERT INTO students (nrp, name)  
VALUES  
    ('31230001', 'Mahasiswa Satu'),  
    ('31230002', 'Mahasiswa Dua')  
ON CONFLICT (nrp) DO NOTHING;
```

```
CREATE INDEX IF NOT EXISTS idx_students_name  
ON students (name);
```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano init/01-schema.sql
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat init/01-schema.sql
CREATE TABLE IF NOT EXISTS students (
    id BIGSERIAL PRIMARY KEY,
    nrp VARCHAR(20) UNIQUE NOT NULL,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO students (nrp, name)
VALUES
    ('31230001', 'Mahasiswa Satu'),
    ('31230002', 'Mahasiswa Dua')
ON CONFLICT (nrp) DO NOTHING;

CREATE INDEX IF NOT EXISTS idx_students_name
ON students (name);

```

4.6 File pgadmin/servers.json

Menulis konfigurasi server agar pgAdmin langsung mendaftarkan server "PostgreSQL Bab 5" setelah login.

```
nano pgadmin/servers.json
```

```

{
  "Servers": {
    "1": {
      "Name": "PostgreSQL Bab 5",
      "Group": "Laboratorium DevSecOps",
      "Host": "postgres-db",
      "Port": 5432,
      "MaintenanceDB": "labdb",
      "Username": "labuser",
      "SSLMode": "prefer"
    }
  }
}

```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano pgadmin/servers.json
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat pgadmin/servers.json
{
  "Servers": {
    "1": {
      "Name": "PostgreSQL Bab 5",
      "Group": "Laboratorium DevSecOps",
      "Host": "postgres-db",
      "Port": 5432,
      "MaintenanceDB": "labdb",
      "Username": "labuser",
      "SSLMode": "prefer"
    }
  }
}
```

Hostname pgAdmin harus menggunakan `postgres-db`, bukan `localhost`, karena pgAdmin berjalan dalam container berbeda; di dalam container pgAdmin, `localhost` berarti container pgAdmin itu sendiri.

4.7 File `scripts/backup.sh`

Menulis script backup yang menggunakan `docker compose exec`, sehingga tidak bergantung pada nama container dinamis. Hasil dump (format custom) diberi timestamp UTC, lalu dibuatkan checksum SHA-256.

```
nano scripts/backup.sh
```

```
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

mkdir -p backup

timestamp="$(date -u +%Y%m%dT%H%M%SZ)"
dump_file="backup/${POSTGRES_DB}-${timestamp}.dump"
checksum_file="${dump_file}.sha256"

docker compose exec -T postgres-db \
    pg_dump \
    --username "$POSTGRES_USER" \
    --dbname "$POSTGRES_DB" \
    --format custom \
    --no-owner \
    --no-privileges \
    > "$dump_file"

test -s "$dump_file"
sha256sum "$dump_file" > "$checksum_file"

echo "[PASS] Backup: $dump_file"
echo "[PASS] Checksum: $checksum_file"
```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano scripts/backup.sh
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat scripts/backup.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

mkdir -p backup

timestamp="$(date -u +%Y%m%dT%H%M%SZ)"
dump_file="backup/${POSTGRES_DB}-${timestamp}.dump"
checksum_file="${dump_file}.sha256"

docker compose exec -T postgres-db \
    pg_dump \
    --username "$POSTGRES_USER" \
    --dbname "$POSTGRES_DB" \
    --format custom \
    --no-owner \
    --no-privileges \
    > "$dump_file"

test -s "$dump_file"
sha256sum "$dump_file" > "$checksum_file"

echo "[PASS] Backup: $dump_file"
echo "[PASS] Checksum: $checksum_file"

```

4.8 File scripts/restore-test.sh

Menulis script restore test yang **tidak menimpa** labdb ; restore dilakukan ke database terpisah labdb_restore_test . Script memverifikasi checksum bila tersedia, membuat ulang database tujuan, menjalankan pg_restore , lalu membuktikan hasil dengan query COUNT(*) .

```
nano scripts/restore-test.sh
```

```

#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

dump_file="${1:-}"
restore_database="${POSTGRES_DB}_restore_test"

if [[ -z "$dump_file" || ! -s "$dump_file" ]]; then
    echo "Penggunaan: $0 backup/nama-file.dump" >&2
    exit 1
fi

if [[ -f "${dump_file}.sha256" ]]; then
    sha256sum --check "${dump_file}.sha256"
fi

docker compose exec -T postgres-db \
    dropdb --username "$POSTGRES_USER" --if-exists "$restore_database"

docker compose exec -T postgres-db \
    createdb --username "$POSTGRES_USER" "$restore_database"

docker compose exec -T postgres-db \
    pg_restore \
    --username "$POSTGRES_USER" \
    --dbname "$restore_database" \
    --no-owner \
    --no-privileges \
    < "$dump_file"

docker compose exec -T postgres-db \
    psql \
    --username "$POSTGRES_USER" \

```

```
--dbname "$restore_database" \  
--command "SELECT COUNT(*) AS total_students FROM students;"
```

```
echo "[PASS] Restore berhasil ke database: $restore_database"
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ nano scripts/restore-test.sh  
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ cat scripts/restore-test.sh  
#!/usr/bin/env bash  
set -Eeuo pipefail  
  
cd "$(dirname "$0")/.."  
  
if [[ ! -f .env ]]; then  
    echo "[FAIL] File .env tidak ditemukan." >&2  
    exit 1  
fi  
  
set -a  
source .env  
set +a  
  
dump_file="${1:-}"  
restore_database="${POSTGRES_DB}_restore_test"  
  
if [[ -z "$dump_file" || ! -s "$dump_file" ]]; then  
    echo "Penggunaan: $0 backup/nama-file.dump" >&2  
    exit 1  
fi  
  
if [[ -f "${dump_file}.sha256" ]]; then  
    sha256sum --check "${dump_file}.sha256"  
fi  
  
docker compose exec -T postgres-db \  
    dropdb --username "$POSTGRES_USER" --if-exists "$restore_database"  
  
docker compose exec -T postgres-db \  
    createdb --username "$POSTGRES_USER" "$restore_database"  
  
docker compose exec -T postgres-db \  
    pg_restore \  
    --username "$POSTGRES_USER" \  
    --dbname "$restore_database" \  
    --no-owner \  
    --no-privileges \  
    < "$dump_file"  
  
docker compose exec -T postgres-db \  
    psql \  
    --username "$POSTGRES_USER" \  
    --dbname "$restore_database" \  
    --command "SELECT COUNT(*) AS total_students FROM students;"  
  
echo "[PASS] Restore berhasil ke database: $restore_database"
```

4.9 Permission dan Validasi File

Memberikan permission eksekusi pada kedua script, lalu memvalidasi daftar file dan model Compose sebelum menjalankan apa pun.

```
cd ~/docker-lab/bab-5
chmod +x scripts/backup.sh scripts/restore-test.sh
```

```
find . -maxdepth 3 -type f | sort
docker compose config
docker compose config --services
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ chmod +x scripts/backup.sh scripts/restore-test.sh
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ find . -maxdepth 3 -type f | sort
./env
./env.example
./gitignore
./compose.yaml
./init/01-schema.sql
./pgadmin/servers.json
./scripts/backup.sh
./scripts/restore-test.sh
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose config
name: bab-5
services:
  pgadmin:
    depends_on:
      postgres-db:
        condition: service_healthy
        required: true
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@example.local
      PGADMIN_DEFAULT_PASSWORD: admin123
    image: dpage/pgadmin4:latest
    networks:
      data-net: null
    ports:
      - mode: ingress
        host_ip: 127.0.0.1
        target: 80
        published: "5050"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: volume
        source: pgadmin-data
        target: /var/lib/pgadmin
        volume: {}
      - type: bind
        source: /home/ubuntu123/docker-lab/bab-5/pgadmin/servers.json
        target: /pgadmin4/servers.json
        read_only: true
        bind: {}
  postgres-db:
    environment:
      POSTGRES_DB: labdb
      POSTGRES_PASSWORD: labpass123
      POSTGRES_USER: labuser
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}
      timeout: 5s
      interval: 5s
      retries: 10
      start_period: 10s
    image: postgres:16-alpine
    networks:
      data-net: null
    ports:
      - mode: ingress
        host_ip: 127.0.0.1
        target: 5432
        published: "5432"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: volume
        source: pg-data
        target: /var/lib/postgresql/data
        volume: {}
      - type: bind
        source: /home/ubuntu123/docker-lab/bab-5/init
        target: /docker-entrypoint-initdb.d
        read_only: true
        bind: {}
networks:
  data-net:
    name: bab-5_data-net
    driver: bridge
volumes:
  pg-data:
```

```

    name: bab-5_pg-data
pgadmin-data:
    name: bab-5_pgadmin-data
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose config --services
postgres-db
pgadmin
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ |

```

4.10 Menjalankan Stack

Menjalankan stack di background, memeriksa status, dan membaca log awal.

```

docker compose up -d
docker compose ps
docker compose logs --tail 100

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose up -d
[+] up 22/22
✓Image dpape/pgadmin4:latest      Pulled      41.6s
✓Network bab-5_data-net           Created     0.3s
✓Volume bab-5_pgadmin-data        Created     0.0s
✓Volume bab-5_pg-data             Created     0.0s
✓Container bab-5-postgres-db-1    Healthy    7.0s
✓Container bab-5-pgadmin-1        Started     7.6s
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ |

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED
STATUS
bab-5-pgadmin-1     dpape/pgadmin4:latest  "/entrypoint.sh"      pgadmin    About a minute ago
o Restarting (1) 3 seconds ago
bab-5-postgres-db-1 postgres:16-alpine     "docker-entrypoint.s..." postgres-db About a minute ago
o Up About a minute (healthy)  127.0.0.1:5432->5432/tcp

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose logs --tail 100
pgadmin-1 | email config is {'CHECK_EMAIL_DELIVERABILITY': False, 'ALLOW_SPECIAL_EMAIL_DOMAINS': [],
pgadmin-1 | 'GLOBALLY_DELIVERABLE': True}
pgadmin-1 | 'admin@example.local' does not appear to be a valid email address. Please reset the PGADM
IN_DEFAULT_EMAIL environment variable and try again.
pgadmin-1 | Validation output: The part after the @-sign is a special-use or reserved name that cannot
t be used with email.
pgadmin-1 | False
pgadmin-1 | email config is {'CHECK_EMAIL_DELIVERABILITY': False, 'ALLOW_SPECIAL_EMAIL_DOMAINS': [],
pgadmin-1 | 'GLOBALLY_DELIVERABLE': True}
pgadmin-1 | 'admin@example.local' does not appear to be a valid email address. Please reset the PGADM
IN_DEFAULT_EMAIL environment variable and try again.
pgadmin-1 | Validation output: The part after the @-sign is a special-use or reserved name that cannot
t be used with email.
pgadmin-1 | False
pgadmin-1 | email config is {'CHECK_EMAIL_DELIVERABILITY': False, 'ALLOW_SPECIAL_EMAIL_DOMAINS': []

```

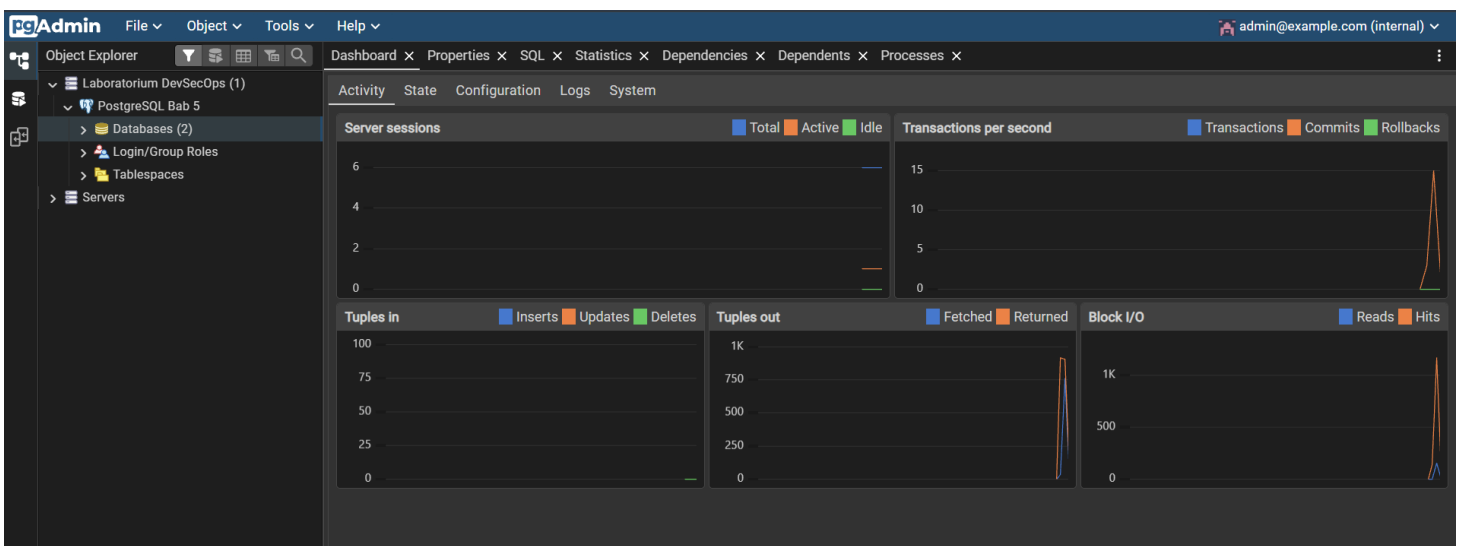
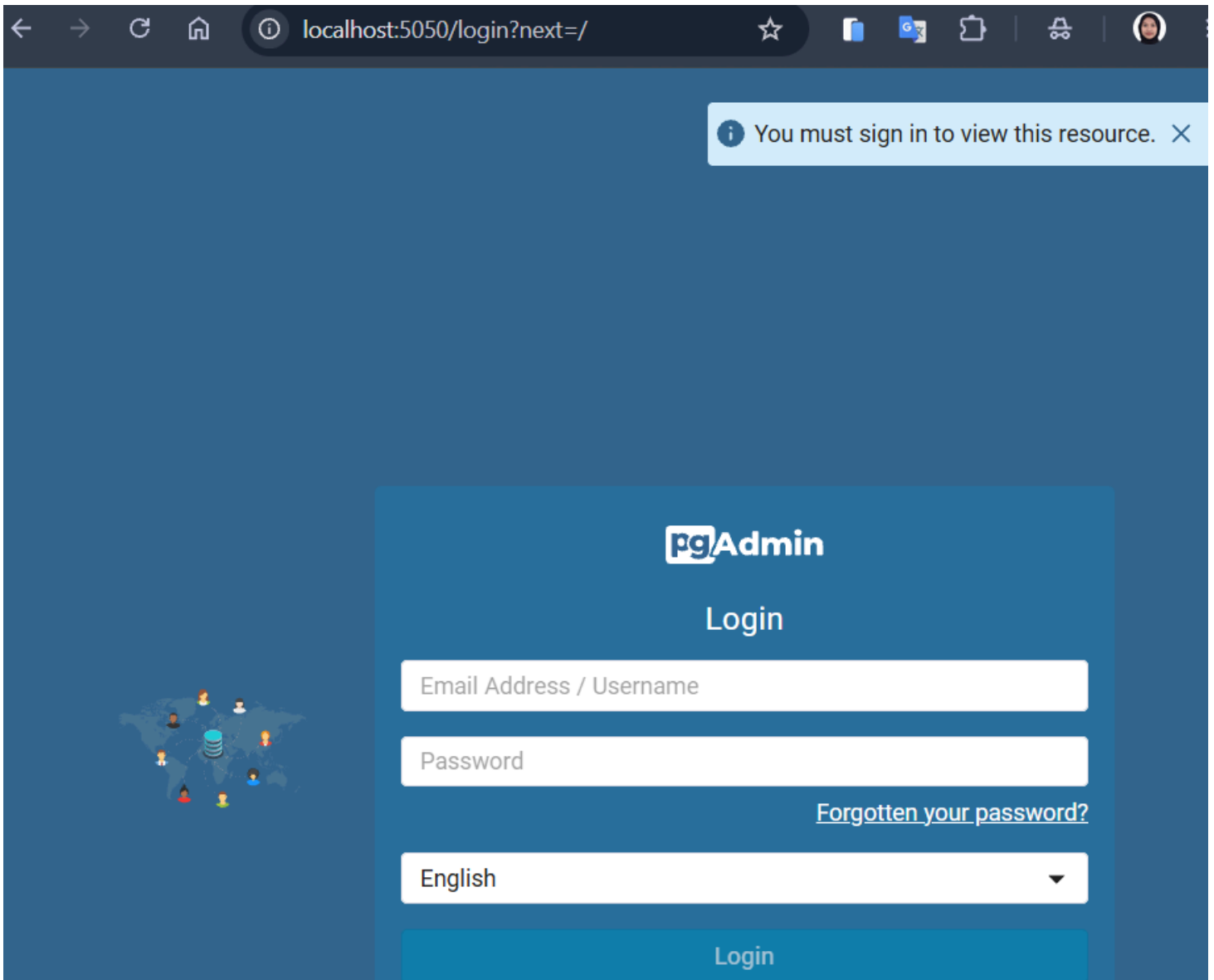
Tunggu sampai PostgreSQL berstatus `healthy` . Akses pgAdmin melalui:

```

http://localhost:5050

```

Login menggunakan nilai PGADMIN_DEFAULT_EMAIL dan PGADMIN_DEFAULT_PASSWORD pada .env , lalu hubungkan server "PostgreSQL Bab 5" dengan memasukkan password PostgreSQL (1abpass123).



4.11 Menguji Schema dan Data Awal

Memeriksa tabel yang terdaftar dan isi data awal dari dalam container PostgreSQL.

```
docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "\dt"
```

```
docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "\dt"
          List of relations
Schema |   Name   | Type  | Owner
-----+-----+-----+-----
public | students | table | labuser
(1 row)
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
 id |   nrp   |      name      |      created_at
----+-----+-----+-----
  1 | 31230001 | Mahasiswa Satu | 2026-10-08 13:44:17.937922+00
  2 | 31230002 | Mahasiswa Dua  | 2026-10-08 13:44:17.937922+00
(2 rows)
```

Keluaran menampilkan tabel `students` dan dua baris data awal (Mahasiswa Satu , Mahasiswa Dua).

4.12 Menguji Persistensi Volume

Menambahkan satu record baru, lalu membuat ulang container **tanpa menghapus volume** untuk membuktikan data tetap ada.

```
docker compose exec -T postgres-db \
  psql -U labuser -d labdb \
  -c "INSERT INTO students(nrp, name) VALUES ('31230003', 'Mahasiswa Tiga') ON CONFLICT DO NOTH;
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
> psql -U labuser -d labdb \
> -c "INSERT INTO students(nrp, name) values
> ('31230003', 'Mahasiswa Tiga') ON CONFLICT DO
> NOTHING;"
INSERT 0 1
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d
labdb -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
 id | nrp | name | created_at
-----+-----+-----+-----
  1 | 31230001 | Mahasiswa Satu | 2026-10-08 13:44:17.937922+00
  2 | 31230002 | Mahasiswa Dua | 2026-10-08 13:44:17.937922+00
  3 | 31230003 | Mahasiswa Tiga | 2026-10-08 14:00:09.070358+00
(3 rows)
```

docker compose down

docker compose up -d

docker compose exec -T postgres-db \

psql -U labuser -d labdb \

-c "SELECT nrp, name FROM students ORDER BY nrp;"

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose down
[+] down 3/3
✓Container bab-5-pgadmin-1      Removed      2.0s
✓Container bab-5-postgres-db-1 Removed      0.6s
✓Network bab-5_data-net        Removed      0.4s
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose up -d
[+] up 3/3
✓Network bab-5_data-net        Created      0.3s
✓Container bab-5-postgres-db-1 Healthy      6.5s
✓Container bab-5-pgadmin-1     Started      7.4s
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d
labdb -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
 id | nrp | name | created_at
-----+-----+-----+-----
  1 | 31230001 | Mahasiswa Satu | 2026-10-08 13:44:17.937922+00
  2 | 31230002 | Mahasiswa Dua | 2026-10-08 13:44:17.937922+00
  3 | 31230003 | Mahasiswa Tiga | 2026-10-08 14:00:09.070358+00
(3 rows)
```

Record 31230003 (Mahasiswa Tiga) tetap tersedia setelah container dibuat ulang, membuktikan data hidup pada volume pg-data, bukan pada container.

4.13 Backup

Menjalankan script backup, memeriksa file dump yang dihasilkan, dan memverifikasi checksum.

./scripts/backup.sh

ls -lh backup/

sha256sum --check backup/*.dump.sha256

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ ./scripts/backup.sh
[PASS] Backup: backup/labdb-20261008T140632Z.dump
[PASS] Checksum: backup/labdb-20261008T140632Z.dump.sha256
```

```

[INFO] CHECKSUM: backup/labdb-20261008T140632Z.dump.sha256
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ ls -lh backup/
total 8.0K
-rw-r--r-- 1 ubuntu123 ubuntu123 3.4K Oct  8 21:06 labdb-20261008T140632Z.dump
-rw-r--r-- 1 ubuntu123 ubuntu123 101 Oct  8 21:06 labdb-20261008T140632Z.dump.sha256

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ sha256sum --check backup/*.dump.sha256
backup/labdb-20261008T140632Z.dump: OK

```

4.14 Restore Test

Memilih dump terbaru lalu menjalankan script restore test ke database terpisah `labdb_restore_test`.

```

latest_dump="$(find backup -maxdepth 1 -name '*.dump' -type f | sort | tail -n 1)"
./scripts/restore-test.sh "$latest_dump"

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ latest_dump="$(find backup -maxdepth 1 -name '*.dump' -
type f | sort | tail -n 1)"
./scripts/restore-test.sh "$latest_dump"
backup/labdb-20261008T140632Z.dump: OK
NOTICE: database "labdb_restore_test" does not exist, skipping
total_students
-----
              3
(1 row)

[PASS] Restore berhasil ke database: labdb_restore_test

```

Memeriksa isi database hasil restore untuk memastikan jumlah dan isi record sesuai database sumber.

```

docker compose exec -T postgres-db \
psql -U labuser -d labdb_restore_test \
-c "SELECT nrp, name FROM students ORDER BY nrp;"

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
psql -U labuser -d labdb_restore_test \
-c "SELECT nrp, name FROM students ORDER BY nrp;"
 nrp      |      name
-----+-----
 31230001 | Mahasiswa Satu
 31230002 | Mahasiswa Dua
 31230003 | Mahasiswa Tiga
(3 rows)

```

Restore berhasil: database `labdb_restore_test` berisi tiga record yang sama dengan `labdb` (Mahasiswa Satu, Dua, dan Tiga).

4.15 Monitoring Dasar

Perintah untuk membuktikan health, versi server, dan penggunaan volume.

```

docker compose ps
docker compose logs postgres-db --tail 100
docker compose exec -T postgres-db pg_isready -U labuser -d labdb
docker compose exec -T postgres-db psql -U labuser -d labdb -c "SELECT version();"
docker volume ls
docker system df -v

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED
STATUS                              PORTS
bab-5-pgadmin-1                     dpape/pgadmin4:latest              "/entrypoint.sh"                      pgadmin    20 minutes ago
Up 19 minutes                        443/tcp, 127.0.0.1:5050->80/tcp
bab-5-postgres-db-1                 postgres:16-alpine                  "docker-entrypoint.s..."            postgres-db 20 minutes ago
Up 19 minutes (healthy)             127.0.0.1:5432->5432/tcp
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose logs postgres-db --tail 100
postgres-db-1 |
postgres-db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
postgres-db-1 |
postgres-db-1 | 2026-10-08 14:01:54.509 UTC [1] LOG:  starting PostgreSQL 16.15 on x86_64-pc-linux-m
usl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
postgres-db-1 | 2026-10-08 14:01:54.510 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
postgres-db-1 | 2026-10-08 14:01:54.521 UTC [1] LOG:  listening on IPv6 address "::", port 5432
postgres-db-1 | 2026-10-08 14:01:54.532 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/
.s.PGSQL.5432"
postgres-db-1 | 2026-10-08 14:01:54.536 UTC [29] LOG:  database system was shut down at 2026-10-08 1
4:01:19 UTC
postgres-db-1 | 2026-10-08 14:01:54.542 UTC [1] LOG:  database system is ready to accept connections
postgres-db-1 | 2026-10-08 14:06:54.637 UTC [27] LOG:  checkpoint starting: time
postgres-db-1 | 2026-10-08 14:06:55.066 UTC [27] LOG:  checkpoint complete: wrote 7 buffers (0.0%);
0 WAL file(s) added, 0 removed, 0 recycled; write=0.405 s, sync=0.007 s, total=0.430 s; sync files=5,
longest=0.003 s, average=0.002 s; distance=0 kB, estimate=0 kB; lsn=0/196E2D0, redo lsn=0/196E298
postgres-db-1 | 2026-10-08 14:12:14.430 UTC [938] FATAL:  database "labdb_restore_test" does not exi
st
postgres-db-1 | 2026-10-08 14:12:36.318 UTC [987] FATAL:  database "labdb_restore_test" does not exi
st
postgres-db-1 | 2026-10-08 14:13:39.023 UTC [1081] FATAL:  database "labdb_restore_test" does not ex
ist
postgres-db-1 | 2026-10-08 14:13:48.567 UTC [1102] FATAL:  database "restore_test" does not exist
postgres-db-1 | 2026-10-08 14:16:54.267 UTC [27] LOG:  checkpoint starting: time
postgres-db-1 | 2026-10-08 14:18:28.685 UTC [27] LOG:  checkpoint complete: wrote 940 buffers (5.7%)
; 1 WAL file(s) added, 0 removed, 0 recycled; write=94.307 s, sync=0.072 s, total=94.418 s; sync file
s=307, longest=0.003 s, average=0.001 s; distance=4319 kB, estimate=4319 kB; lsn=0/1DA6080, redo lsn=
0/1DA6048

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db pg_isready -U labuse
r -d labdb
/var/run/postgresql:5432 - accepting connections
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d l
abdb -c "SELECT version();"
          version
-----
PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
(1 row)

```

```

ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker volume ls
DRIVER      VOLUME NAME
local       bab-3_pg-data
local       bab-5_pg-data
local       bab-5_pgadmin-data
local       data-vol
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker system df -v
Images space usage:

REPOSITORY          TAG             IMAGE ID        CREATED         SIZE      SHARED SIZE  UNIQUE SIZE   C
ONTAINERS
bab-4-flask-app     latest         6dba7faa4dab   6 days ago     211MB     117.5MB      93.89MB      0
bab-3-app           latest         e016c132804e   2 weeks ago    220MB     117.5MB      102.6MB      1
httpd               2.4-alpine     4e585da9d012   2 weeks ago    97.4MB     0B           97.37MB      0
dpage/pgadmin4      latest         c332c5f6dfba   3 weeks ago    791MB     12.92MB      778MB        1
pens-web            1.0            04812a43d943   4 weeks ago    72.2MB     2B           72.17MB      1
nginx               alpine         72ba65eb42c1   5 weeks ago    103MB     12.92MB      90.02MB      1
postgres            16-alpine     cf78e76683b9   7 weeks ago    420MB     12.92MB      406.8MB      2
ubuntu              22.04         2edbbc5dc405   8 weeks ago    119MB     0B           119.4MB      0
alpine              3.20          d9e853e87e55   5 months ago   12.2MB     0B           12.18MB      0
hello-world         latest         5dd0d3e6e255   6 months ago   25.9kB     0B           25.87kB      1
nginx               1.26          41b194461e4b   20 months ago  282MB     0B           281.6MB      0

Containers space usage:

CONTAINER ID   IMAGE                                COMMAND                                     LOCAL VOLUMES   SIZE      CREATED
STATUS        NAMES
acb45c5e3df6   dpage/pgadmin4:latest              "/entrypoint.sh"                                     1              5.68MB    22 minutes
ago Up 22 minutes   bab-5-pgadmin-1
2849c8d7b352   postgres:16-alpine                 "docker-entrypoint.s..."                         1              20.5kB    22 minutes
ago Up 22 minutes (healthy)   bab-5-postgres-db-1
be882eeb9904   nginx:alpine                       "/docker-entrypoint..."                          0              49.2kB    2 weeks ago
Exited (0) 2 weeks ago   bab-3-web-1
42e2eb56befb   bab-3-app                           "unicorn --bind 0.0..."                          0              8.19kB    2 weeks ago
Exited (0) 2 weeks ago   bab-3-app-1
c6e00eec8a12   postgres:16-alpine                 "docker-entrypoint.s..."                         1              16.4kB    2 weeks ago
Exited (0) 2 weeks ago   bab-3-db-1
8ae61443a638   pens-web:1.0                       "/docker-entrypoint..."                          0              77.8kB    4 weeks ago
Exited (0) 4 weeks ago   pens-app
66d04ee974df   hello-world                         "/hello"                                             0              4.1kB     4 weeks ago
Exited (0) 4 weeks ago   goofy_proskuriakova

Local Volumes space usage:

VOLUME NAME    LINKS    SIZE
bab-3_pg-data  1        48.21MB
data-vol       0        1.189kB
bab-5_pg-data  1        72.92MB
bab-5_pgadmin-data  1        201.9kB

Build cache usage: 709.3MB

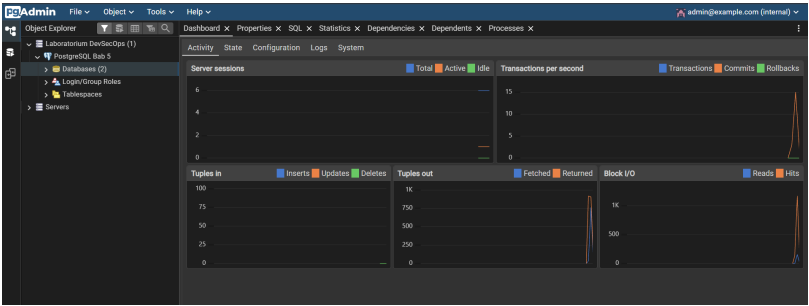
CACHE ID       CACHE TYPE    SIZE      CREATED         LAST USED        USAGE    SHARED
jhxew3f5z7d6   regular       12.1MB    4 weeks ago    4 weeks ago     1        true
2j6yyeawatl4   regular       7.04MB    4 weeks ago    4 weeks ago     1        true
ppowmthqe4x1   regular       8.82kB    4 weeks ago    4 weeks ago     1        true
twycp7zkejf5   regular       13.2kB    4 weeks ago    4 weeks ago     1        true

```

pg_isready mengonfirmasi server menerima koneksi, SELECT version() menunjukkan versi PostgreSQL yang berjalan, dan docker volume ls menampilkan pg-data serta pgadmin-data .

4.16 Checklist PASS

No	Checklist PASS	Evidence
1	docker compose config tidak menghasilkan galat	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ docker compose config name: bab-5 services: pgadmin: depends_on: postgres-db: condition: service_healthy required: true environment: PGADMIN_DEFAULT_EMAIL: admin@example.local PGADMIN_DEFAULT_PASSWORD: admin123 image: dpape/pgadmin4:latest networks: data-net: null ports: - mode: ingress host_ip: 127.0.0.1 target: 80 published: "5050" protocol: tcp restart: unless-stopped volumes: - type: volume source: pgadmin-data target: /var/lib/pgadmin volume: {} - type: bind source: /home/ubuntu123/docker-lab/bab-5/pgadmin/servers.json target: /pgadmin4/servers.json read_only: true bind: {} postgres-db: environment: POSTGRES_DB: labdb POSTGRES_PASSWORD: labpass123 POSTGRES_USER: labuser healthcheck: test: - CMD-SHELL - pg_isready -U \${POSTGRES_USER} -d \${POSTGRES_DB} timeout: 5s interval: 5s retries: 10 start_period: 10s image: postgres:16-alpine networks: data-net: null ports: - mode: ingress host_ip: 127.0.0.1 target: 5432 published: "5432" protocol: tcp restart: unless-stopped volumes: - type: volume source: pg-data target: /var/lib/postgresql/data volume: {} - type: bind source: /home/ubuntu123/docker-lab/bab-5/init target: /docker-entrypoint-initdb.d read_only: true bind: {} networks: data-net: name: bab-5_data-net driver: bridge volumes: pg-data: name: bab-5_pg-data pgadmin-data: name: bab-5_pgadmin-data</pre>
2	PostgreSQL berstatus healthy	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ docker compose ps NAME IMAGE PORTS COMMAND SERVICE CREATED STATUS bab-5-pgadmin-1 dpape/pgadmin4:latest "/entrypoint.sh" pgadmin About a minute ago o Restarting (1) 3 seconds ago bab-5-postgres-db-1 postgres:16-alpine "docker-entrypoint.s..." postgres-db About a minute ago o Up About a minute (healthy) 127.0.0.1:5432->5432/tcp</pre>
3	Tabel students dan data awal tersedia	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ docker compose exec -T postgres-db \ psql -U labuser -d labdb \ -c "\dt" List of relations Schema Name Type Owner -----+-----+-----+----- public students table labuser (1 row)</pre>

No	Checklist PASS	Evidence
4	pgAdmin dapat terhubung menggunakan hostname postgres-db	
5	Data tetap tersedia setelah docker compose down dan up	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;" id nrp name created_at ---+---+---+----- 1 31230001 Mahasiswa Satu 2026-10-08 13:44:17.937922+00 2 31230002 Mahasiswa Dua 2026-10-08 13:44:17.937922+00 3 31230003 Mahasiswa Tiga 2026-10-08 14:00:09.070358+00 (3 rows)</pre>
6	Backup dump berukuran lebih dari nol	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ ls -lh backup/ total 8.0K -rw-r--r-- 1 ubuntu123 ubuntu123 3.4K Oct 8 21:06 labdb-20261008T140632Z.dump -rw-r--r-- 1 ubuntu123 ubuntu123 101 Oct 8 21:06 labdb-20261008T140632Z.dump.sha256</pre>
7	Checksum backup valid	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ sha256sum --check backup/*.dump.sha256 backup/labdb-20261008T140632Z.dump: OK</pre>
8	Restore berhasil pada database labdb_restore_test	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ latest_dump="\$(find backup -maxdepth 1 -name '*.dump' -type f sort tail -n 1)" ./scripts/restore-test.sh "\$latest_dump" backup/labdb-20261008T140632Z.dump: OK NOTICE: database "labdb_restore_test" does not exist, skipping total_students ----- 3 (1 row) [PASS] Restore berhasil ke database: labdb_restore_test</pre>
9	Jumlah dan isi record hasil restore sesuai database sumber	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ docker compose exec -T postgres-db \ psql -U labuser -d labdb_restore_test \ -c "SELECT nrp, name FROM students ORDER BY nrp;" nrp ----+--- 31230001 Mahasiswa Satu 31230002 Mahasiswa Dua 31230003 Mahasiswa Tiga (3 rows)</pre>
10	Credential dan dump tidak masuk repository Git	<pre>ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5\$ cat .gitignore .env backup/*.dump backup/*.sha256 backup/*.log</pre>

Seluruh butir checklist terpenuhi dengan status PASS .

4.17 Cleanup

Perintah cleanup pertama menghentikan dan menghapus container serta network proyek tetapi **mempertahankan** data pada volume.

```
cd ~/docker-lab/bab-5
docker compose down
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose down
[+] down 3/3
✓Container bab-5-pgadmin-1      Removed      2.1s
✓Container bab-5-postgres-db-1 Removed      0.6s
✓Network bab-5_data-net        Removed      0.3s
```

Menghapus database hasil restore test:

```
docker compose up -d postgres-db
docker compose exec -T postgres-db \
    dropdb -U labuser --if-exists labdb_restore_test
```

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ docker compose up -d postgres-db
docker compose exec -T postgres-db \
    dropdb -U labuser --if-exists labdb_restore_test
[+] up 2/2
✓Network bab-5_data-net        Created      0.3s
✓Container bab-5-postgres-db-1 Started      1.0s
```

Perintah berikut untuk menghapus volume sekaligus seluruh data PostgreSQL dan pgAdmin.

```
docker compose down -v
```

4.18 Struktur File Akhir

Susunan file proyek setelah seluruh langkah selesai sebagai dokumentasi hasil akhir.

```
ubuntu123@DESKTOP-7A8ST00:~/docker-lab/bab-5$ find . -maxdepth 3 -type f ! -path "./backup/*" | sort
./env
./env.example
./gitignore
./compose.yaml
./init/01-schema.sql
./pgadmin/servers.json
./scripts/backup.sh
./scripts/restore-test.sh
```

4.19 Ringkasan Hasil Eksekusi

Perintah	Hasil / Catatan	Status
docker compose config	Model Compose valid; hanya postgres-db dan pgadmin dengan published port loopback	PASS
docker compose config --services	Menghasilkan pgadmin , postgres-db	PASS
docker compose up -d	2 container berjalan pada network data-net	PASS
docker compose ps	postgres-db healthy , pgadmin running	PASS

Perintah	Hasil / Catatan	Status
<code>psql -c "\dt"</code>	Tabel <code>students</code> terdaftar	PASS
<code>SELECT ... FROM students</code>	Dua data awal tampil	PASS
<code>INSERT 31230003 + down/up</code>	Record tetap tersedia setelah container dibuat ulang	PASS
<code>./scripts/backup.sh</code>	Dump format custom + checksum dibuat di backup/	PASS
<code>sha256sum --check</code>	Checksum valid (OK)	PASS
<code>./scripts/restore-test.sh</code>	Restore sukses ke <code>labdb_restore_test</code>	PASS
<code>SELECT</code> pada <code>labdb_restore_test</code>	Jumlah dan isi record sesuai database sumber	PASS
<code>pg_isready / SELECT version()</code>	Server menerima koneksi; versi PostgreSQL terverifikasi	PASS
<code>docker compose down</code>	2 container dan 1 network terhapus, volume dipertahankan	PASS

5. Verifikasi

Skenario Pengujian	Hasil
PostgreSQL berjalan dengan volume <code>pg-data</code>	PASS
Tabel <code>students</code> dibuat otomatis dari init script	PASS
pgAdmin dapat login dan terkoneksi ke database	PASS
Data tetap tersedia setelah <code>docker compose down</code> dan <code>up</code>	PASS
Backup menghasilkan file dump di direktori host	PASS
Checksum backup valid	PASS
Restore sudah diuji ke database terpisah, bukan hanya diasumsikan berhasil	PASS
Jumlah dan isi record hasil restore sesuai database sumber	PASS

Skenario Pengujian	Hasil
Credential dan dump tidak masuk repository Git	PASS

6. Evaluasi dan Latihan Mandiri

1. Mengapa init script tidak dijalankan ulang saat volume lama masih ada?
2. Apa risiko menaruh password database pada `docker-compose.yml` ?
3. Bagaimana cara membuktikan backup dapat dipulihkan?
4. Apa bedanya logical backup `pg_dump` dan backup filesystem volume mentah?
5. Apa dampak `docker compose down -v` terhadap database?

6.1 Jawaban

1. Image PostgreSQL hanya menjalankan berkas di `/docker-entrypoint-initdb.d`. Ketika direktori data `/var/lib/postgresql/data` masih kosong. Volume `pd-data` yang sudah berisi PostgreSQL menandakan insialisasi awal sudah selesai, sehingga entrypoint melewati tahap init dan langsung menjalankan server. Jika init script dieksekusi ulang pada data yang sudah ada, operasi seperti `CREATE TABLE` atau `INSERT` dapat bertabrakan atau menimpa data nyata.
2. Risiko Utamanya adalah kebocoran credential. Umumnya `docker-compose.yml` ikut tercomit ke git sehingga password akan ikut masuk ke version control beserta riwayatnya. Password juga akan terekspose lewat `docker compose config`, `docker inspect`, dan `environment` proses container, sehingga orang lain yang memiliki akses repositori dapat membacanya dalam bentuk plaintext. Credential sebaiknya ditaruh di `.env` yang di `.gitignore`.
3. Backup dibuktikan dengan verifikasi integritas dump menggunakan checksum SHA-256, lalu dilakukan restore ke basis data terpisah agar tidak menimpa data sumber. Selanjutnya, hasil restore diverifikasi menggunakan kueri `SELECT COUNT(*)` untuk dibandingkan dengan basis data asal, serta mencatat durasi restore sebagai data RTO.
4. logical backup `pg_dump` membaca objek database melalui koneksi SQL dan menghasilkan berkas berisi DDL dan data pada snapshot transaksional. Sedangkan Backup filesystem volume mentah menyalin langsung berkas data PostgreSQL di tingkat berkas/block.
5. Dampak dari menggunakan perintah `docker compose down -v` adalah menghapus container, network proyek, dan karena menggunakan flag `-v` maka seluruh database dan record ikut terhapus. Data tidak dapat dipulihkan kecuali dari backup.

7. Analisis Hasil

7.1 Masalah yang Muncul dan Cara Mendiagnosisnya

Gejala yang Muncul	Langkah Diagnosis	Akar Penyebab	Tindakan yang Diterapkan
Login pgAdmin di <code>http://localhost:5050</code> gagal; email <code>admin@example.local</code> ditolak/bersalah format meskipun password benar	Membuka <code>http://localhost:5050</code> , mencoba login dengan <code>PGADMIN_DEFAULT_EMAIL</code> dan <code>PGADMIN_DEFAULT_PASSWORD</code> dari <code>.env</code> , lalu memeriksa ulang nilai <code>.env</code> dan log container <code>pgadmin</code>	Validator email pgAdmin menolak <code>.local</code> karena termasuk special-use/reserved domain yang tidak valid;	Mengganti <code>PGADMIN_DEFAULT_EMAIL</code> pada <code>.env</code> menjadi <code>admin@example.com</code> , lalu menjalankan ulang stack dengan <code>docker compose up -d</code> dan login kembali dengan email beserta password baru

7.2 Risiko Keamanan pada Bab Ini

- File `.env` berisiko bocor jika ter-commit ke Git.
- Isi `.env` tetap dapat terlihat melalui perintah `docker inspect` atau `docker compose config`.
- Volume bukan backup karena tetap rentan terhapus perintah `down -v` atau mengalami kerusakan storage.
- Script SQL awal hanya dieksekusi sekali saat volume masih kosong.
- File dump di host yang sama rentan hilang total jika terjadi kerusakan disk.

7.3 Rekomendasi yang Disarankan untuk Environment Production

1. Mengganti `.env` dengan secret manager atau Docker secret ber-permission minimalis dan lakukan rotasi credential.
2. Mengelola perubahan skema dengan migration tool ternversi yang mendukung uji rollback.
3. Menutup akses published port PostgreSQL jika tidak diperlukan, aktifkan TLS, dan batasi `listen_addresses`.
4. Menerapkan batasan CPU/RAM pada container serta gunakan connection pooler untuk efisiensi koneksi.

5. Memisahkan peran (role) basis data untuk migrasi, runtime, backup, dan observability.
6. Memantau performa database secara real-time dan hubungkan alert dengan prosedur respon insiden.

8. Kesimpulan

Praktikum Bab 5 berhasil menjalankan PostgreSQL dan pgAdmin sebagai container dengan konfigurasi berbasis environment variable, volume persisten, dan init script. Tabel students beserta data awalnya dibuat otomatis pada inisialisasi, dan pgAdmin dapat terhubung menggunakan hostname service postgres-db.

Eksperimen membuktikan data pada volume pg-data tetap persisten saat container dibuat ulang (down & up), namun akan hilang total jika dihentikan menggunakan flag -v (down -v). File dump dari pg_dump tervalidasi via checksum SHA-256 dan berhasil di-restore ke database terpisah labdb_restore_test.

9. Catatan Penggunaan AI

Penggunaan AI dalam laporan ini untuk menyusun format laporan, Melakukan generate peta konsep, dan media diskusi untuk memahami materi. Adapun analisis hasil, kesimpulan, dan latihan mandiri dibuat oleh saya sendiri.